

대규모 스킬 코퍼스 라우팅: BM25 vs 하이브리드 검색

1000+ 도구에서 올바른 스킬 찾기

2i

2026-07-29

초록

에이전트가 다룰 수 있는 도구(스킬, 함수, API)의 목록이 수백 개를 넘어가면, 모든 도구 설명을 매 요청마다 프롬프트에 통째로 넣는 방식은 더 이상 성립하지 않는다. 그래서 실무에서는 사용자 요청과 가장 관련 있는 소수의 도구만 먼저 검색(retrieve)한 뒤, 그 후보만 모델에 보여주는 2단계 구조를 쓴다. 이 논문은 그 검색 단계 자체를 실험 대상으로 삼는다. 1000 개 이상의 항목으로 구성된, 완전히 공개된 방법으로 생성한 합성 도구 코퍼스를 만들고, 어휘 기반 검색(BM25), 의미 기반에 가까운 검색(문자 n-gram TF-IDF 코사인 유사도), 그리고 둘을 상호순위융합(Reciprocal Rank Fusion, RRF)으로 결합한 하이브리드 검색을 같은 질의 집합에 대해 실측했다. 질의는 정답이 있는 180개의 긍정 질의(코퍼스 항목을 동의어로 바꿔서 표면 어휘를 다르게 만든 것)와, 정답이 존재하지 않아야 하는 41개의 하드 네거티브 질의(서로 다른 도메인의 동작과 대상을 억지로 엮은 질의)로 구성했다. 실측 결과 하이브리드 RRF는 Recall@1에서 0.508로 BM25(0.508)와 사실상 동률, TF-IDF 의사-의미 검색(0.517)보다 근소하게 낮았고, Recall@5(0.683)와 MRR(0.603)에서는 두 단일 검색기보다 오히려 낮았다. 반면 오답 회피(하드 네거티브에 대한 점수 게이트 기반 기권율)에서는 BM25 26.8%, TF-IDF 31.7%, 하이브리드 RRF 12.2%로, 하이브리드가 오히려 가장 나빴다. 이 논문은 '하이브리드가 항상 이긴다'는 흔한 통념을 검증 없이 반복하지 않는다. 이 특정 실험 설계에서는 RRF가 순위 정보만 쓰고 절대 점수 크기를 버리기 때문에, 근거가 약한 상위 1위 결과도 각 개별 검색기에서 순위가 높기만 하면 융합 점수가 높게 나와 기권 게이트를 무력화한다는 것을 데이터로 보여준다. 코퍼스와 질의 생성 규칙, 세 검색기의 구현, 계산에 쓰인 실제 수치는 모두 이 논문에 재현 가능한 형태로 공개한다.

목차

1 요약	3
2 배경: 도구가 수천 개가 되면 검색이 곧 정확도다	3
3 방법: BM25, 유사-의미 검색, 그리고 점수 게이트가 있는 RRF	3
3.1 BM25 (어휘 검색)	3
3.2 유사-의미 검색: 문자 n-gram TF-IDF 코사인 유사도	4
3.3 하이브리드: 상호순위융합(RRF)	4
3.4 점수 게이트를 통한 기권(abstain)	4
4 데이터와 실험 설정: 완전히 공개된 합성 코퍼스	4
4.1 코퍼스 생성	4
4.2 질의 집합	4
4.3 재현성과 라이선스	5
5 결과	5
5.1 전체 성능 비교	5
5.2 오답 회피(기권) 성능	6
6 한계와 캐비엣	6
7 데이터와 재현	7
8 참고문헌	7

1 요약

에이전트가 다룰 수 있는 도구(스킬, 함수, API)의 목록이 수백 개를 넘어가면, 모든 도구 설명을 매 요청마다 프롬프트에 통째로 넣는 방식은 더 이상 성립하지 않는다. 그래서 실무에서는 사용자 요청과 가장 관련 있는 소수의 도구만 먼저 검색(retrieve)한 뒤, 그 후보만 모델에 보여주는 2단계 구조를 쓴다. 이 논문은 그 검색 단계 자체를 실험 대상으로 삼는다. 1000개 이상의 항목으로 구성된, 완전히 공개된 방법으로 생성한 합성 도구 코퍼스를 만들고, 어휘 기반 검색(BM25), 의미 기반에 가까운 검색(문자 n-gram TF-IDF 코사인 유사도), 그리고 둘을 상호순위융합(Reciprocal Rank Fusion, RRF)으로 결합한 하이브리드 검색을 같은 질의 집합에 대해 실측했다. 질의는 정답이 있는 180개의 긍정 질의(코퍼스 항목을 동의어로 바꿔 써서 표면 어휘를 다르게 만든 것)와, 정답이 존재하지 않아야 하는 41개의 하드 네거티브 질의(서로 다른 도메인의 동작과 대상을 억지로 엮은 질의)로 구성했다. 실측 결과 하이브리드 RRF는 Recall@1에서 0.508로 BM25(0.508)와 사실상 동률, TF-IDF 의사-의미 검색(0.517)보다 근소하게 낮았고, Recall@5(0.683)와 MRR(0.603)에서는 두 단일 검색기보다 오히려 낮았다. 반면 오답 회피(하드 네거티브에 대한 점수 게이트 기반 기권율)에서는 BM25 26.8%, TF-IDF 31.7%, 하이브리드 RRF 12.2%로, 하이브리드가 오히려 가장 나빴다. 이 논문은 “하이브리드가 항상 이긴다”는 흔한 통념을 검증 없이 반복하지 않는다. 이 특정 실험 설계에서는 RRF가 순위 정보만 쓰고 절대 점수 크기를 버리기 때문에, 근거가 약한 상위 1위 결과도 각 개별 검색기에서 순위가 높기만 하면 융합 점수가 높게 나와 기권 게이트를 무력화한다는 것을 데이터로 보여준다. 코퍼스와 질의 생성 규칙, 세 검색기의 구현, 계산에 쓰인 실제 수치는 모두 이 논문에 재현 가능한 형태로 공개한다.

2 배경: 도구가 수천 개가 되면 검색이 곧 정확도다

거대언어모델을 에이전트로 쓸 때 가장 먼저 부딪히는 확장성 문제는 도구 목록이다. 도구가 열 개, 스무 개일 때는 전체 설명을 시스템 프롬프트에 넣어도 별문제가 없다. 그러나 실제 배포 환경에서 에이전트가 접근할 수 있는 능력의 수는 빠르게 수백, 수천 개로 불어난다. 사내 업무 자동화 플랫폼이 부서마다 스킬을 추가하고, 개인화된 어시스턴트가 사용자마다 다른 연동을 켜고, 마켓플레이스형 에이전트 생태계가 서드파티 도구를 계속 등록하는 식이다. 이 규모에서는 모든 도구 설명을 프롬프트에 다 넣는 것 자체가 불가능해진다. 컨텍스트 길이 제한을 넘어서기도 하지만, 그보다 먼저 모델이 수천 개의 후보 중에서 올바른 한두 개를 골라내는 정확도 자체가 무너진다. 관련 없는 도구 설명이 많을수록 모델은 이름이 비슷하거나 설명이 그럴듯한 엉뚱한 도구를 고르는 실수를 더 자주 저지른다.

그래서 실무에서 널리 쓰이는 해법은 도구 목록 전체를 매번 보여주는 대신, 사용자 요청을 질의로 삼아 후보 도구를 먼저 검색하고, 상위 소수만 모델에 넘기는 2단계 구조다. 이 구조에서는 최종 선택의 품질이 1단계 검색기의 품질에 의해 상한선이 그어진다. 검색이 정답 도구를 후보군에서 빼먹으면, 아무리 뛰어난 모델도 그 도구를 선택할 수 없다. 반대로 검색이 관련 없는 도구를 후보 상위에 많이 올리면, 모델의 오선택 확률이 그만큼 높아진다. 도구 검색을 연구하는 선행 작업들(대표적으로 대규모 API 카탈로그에서 모델이 직접 API를 호출하도록 학습시킨 시도들, 그리고 도구 사용을 자기지도로 학습시키는 시도들)은 대체로 이 검색 단계를 암묵적 전제로 깔고 모델의 사용 능력 자체에 집중해왔다. 이 논문은 반대로 그 전제, 즉 “검색이 잘 될 것”이라는 가정 자체를 실험 대상으로 삼는다.

특히 세 가지 질문에 답하고자 한다. 첫째, 순수 어휘 매칭(사용자 요청 문장과 도구 설명 사이의 단어 중복도를 계산하는 방식)만으로 충분한가, 아니면 표면 어휘가 다른 요청도 잡아낼 수 있는 의미 기반 검색이 꼭 필요한가. 둘째, 둘을 섞은 하이브리드 검색이 실제로 각각의 단일 검색기보다 나은가, 아니면 “섞으면 무조건 좋아진다”는 것은 검증 없는 통념인가. 셋째, 정답이 존재하지 않는 요청(코퍼스에 없는 능력을 요구하는 요청)에 대해 검색기가 “모르겠다”고 기권할 수 있는가. 세 번째 질문은 실무에서 특히 중요하다. 에이전트가 지원하지 않는 기능을 요청받았을 때, 검색기가 억지로 가장 비슷해 보이는 엉뚱한 도구를 1위로 올려버리면 모델은 그 도구를 실제로 호출해버릴 위험이 있다. 반대로 검색기가 관련도 낮은 결과에는 점수를 낮게 매겨 기권 신호를 낼 수 있다면, 상위 애플리케이션은 “지원하지 않는 요청입니다”라고 정직하게 답할 기회를 얻는다.

3 방법: BM25, 유사-의미 검색, 그리고 점수 게이트가 있는 RRF

3.1 BM25 (어휘 검색)

BM25는 정보검색에서 가장 널리 쓰이는 어휘 기반 순위화 함수다. 질의 q 의 각 항 t 에 대해 문서 d 의 점수를 다음과 같이 계산한다.

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1} \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right) \cdot \$$$

여기서 $f(t, d)$ 는 항 t 가 문서 d 에 등장한 횟수, $|d|$ 는 문서 길이, avgdl 은 코퍼스 평균 문서 길이다. IDF는 역문서 빈도로, 코퍼스 전체에서 드물게 등장하는 단어일수록 가중치를 높게 준다. 이 논문에서는 외부 라이브러리에 의존하지 않고 이

식을 직접 구현했다 (\$k_1=1.5\$, \$b=0.75\$, 표준 관행값). 토큰화는 소문자 변환 후 영숫자 시퀀스만 추출하는 단순한 방식을 썼으며, 형태소 분석이나 어간 추출(stemming)은 적용하지 않았다. 이 선택은 의도적이다. 어간 추출을 적용하면 “transcribe”와 “transcribing”이 같은 항으로 묶여 BM25가 형태 변화에 강해지는데, 이는 이 논문이 검증하려는 “어휘 매칭의 한계”를 인위적으로 완화시켜 하이브리드 검색의 필요성을 실제보다 낮게 보이게 만들 수 있다.

3.2 유사-의미 검색: 문자 n-gram TF-IDF 코사인 유사도

이 실험 환경은 사내망을 통해 pypi와 github에는 접근할 수 있지만 huggingface.co에서 사전학습 임베딩 모델을 내려받을 수 없는 조건에 묶여 있었다. 그래서 진짜 의미 임베딩 대신, 문서와 질의를 3~5글자 문자 n-gram(단어 경계를 인식하는 char_wb 방식) TF-IDF 벡터로 변환하고 코사인 유사도로 순위를 매기는 두 번째 검색기를 구현했다. 이 방식은 진짜 의미 벡터가 아니다. “resize”와 “scale”처럼 완전히 다른 단어로 표현된 동의어는 문자 단위로 겹치는 부분이 거의 없어 잡아내지 못한다. 대신 이 방식이 실제로 잘 잡는 것은 형태론적 변형과 부분 문자열 유사성이다. “transcribe”와 “transcribing”, “encrypt”와 “encrypting”처럼 어간이 같고 어미만 다른 표현, 그리고 띄어쓰기나 복수형 차이 같은 표면적 변주다. 이 논문은 이 검색기를 “덴스(dense)”라고 과장하지 않고, 순수 단어 단위 어휘 매칭(BM25)과는 다른 종류의 겹침 신호를 제공하는 두 번째 다리(leg)로 정직하게 자리매김한다. 진짜 사전학습 의미 임베딩과의 비교는 이 논문의 범위 밖이며, 한계 절에서 다시 짚는다.

3.3 하이브리드: 상호순위융합(RRF)

두 검색기의 결과를 합치는 방법으로 상호순위융합(Reciprocal Rank Fusion, RRF)을 썼다. RRF는 절대 점수의 크기나 분포를 신경 쓰지 않고, 각 검색기가 매긴 순위만을 이용해 다음과 같이 문서별 융합 점수를 계산한다.

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

여기서 \$R\$은 검색기 집합(BM25, TF-IDF 두 개), \$\text{rank}_r(d)\$는 검색기 \$r\$이 문서 \$d\$에 매긴 순위(1부터 시작), \$k\$는 순위 초반의 편향을 완화하는 상수다(이 논문에서는 문헌에서 흔히 쓰이는 \$k=60\$을 그대로 사용했고 별도로 튜닝하지 않았다). 두 검색기 각각 상위 50개까지만 융합에 반영했다(융합 깊이 50).

3.4 점수 게이트를 통한 기권(abstain)

세 검색기 모두 “이 질의에는 답이 없다”고 판단할 수 있어야 한다는 요구를 만족시키기 위해, 각 검색기의 1위 점수에 임계값을 씌우는 점수 게이트를 뒀다. 임계값은 정답이 있는 질의 중 일부를 따로 떼어낸 보정(calibration) 집합에서, 그 집합의 1위 점수 분포의 하위 10번째 백분위로 정했다. 즉 “진짜 정답이 있는 질의의 약 90%는 이 임계값을 넘을 것”이라는 기준으로 임계값을 정하고, 이 임계값을 그대로 (1) 보정에 쓰이지 않은 나머지 정답 질의와 (2) 정답이 없어야 하는 하드 네거티브 질의에 적용해 결과를 관찰했다. 이 절차는 검색기마다 독립적으로 반복했다. BM25 점수, TF-IDF 코사인 유사도, RRF 융합 점수는 스케일이 서로 다르기 때문에 임계값도 검색기마다 다르게 나온다는 점을 미리 밝혀둔다.

4 데이터와 실험 설정: 완전히 공개된 합성 코퍼스

4.1 코퍼스 생성

이 실험은 실제 사내 도구 목록이 아니라, 재현 가능하고 완전히 공개된 절차로 만든 합성 코퍼스를 쓴다. 20개 영역(여행, 금융, 이미지, 영상, 오디오, 텍스트, 이메일, 캘린더, 쇼핑, 건강, 날씨, 지도, 소셜, 생산성, 보안, 번역, 코드, 데이터, 사물인터넷, 음악) 각각에 10개의 동작(예: 검색, 예약, 변환, 압축)과 10개의 대상 명사(예: 항공권, 이미지, 이메일)를 정의했다. 각 영역 안에서는 동작과 대상을 전수 교차하지 않고, 순환 오프셋(cyclic offset) 방식으로 짝지었다. 즉 오프셋을 0부터 9까지 바꿔가며 동작 목록을 회전시켜 대상 목록과 짝짓는 방식으로, 영역당 100개(10개 오프셋 x 10개 대상), 전체 20개 영역에서 2000개의 항목을 만들었다. 각 항목은 이름(동작과 대상을 이어붙인 식별자)과 설명(다섯 개의 문장 틀 중 하나에 동작·대상·영역명을 채운 한 줄 설명)으로 구성했다. 예를 들어 “normalize_speech_samples” 항목의 설명은 “Automate the ability to normalize speech samples for audio workflows.”이다. 코퍼스 생성은 시드 42로 고정했고, 항목 순서를 섞는 것을 포함해 완전히 결정론적이다. 이 방법은 도메인 지식이 없어도 그대로 재구현할 수 있도록 이 논문에 전부 공개한다.

4.2 질의 집합

정답이 있는 긍정 질의 180개는 코퍼스에서 무작위로 뽑은 항목마다 하나씩 만들었다. 이때 항목의 원래 동작 단어를 그대로 쓰지 않고, 75%의 확률로 사전에 정의한 동의어 사전에서 다른 표현으로 바꿔치기했다(예: “compress”를 “shrink”나 “reduce

the file size of"로). 그 위에 여섯 가지 자연어 질문 틀 중 하나를 무작위로 씌워 "Can you shrink the following podcast episodes?"와 같은 실제 사용자 발화에 가까운 문장을 만들었다. 이렇게 만든 질의는 정답 항목의 설명과 표면 문자열이 상당히 다르면서도 의미적으로는 같은 요청이 되도록 설계했다. 180개 중 60개는 점수 게이트 임계값 보정에, 나머지 120개는 검색 성능(Recall@1, Recall@5, MRR) 측정에 썼다.

정답이 없어야 하는 하드 네거티브 질의는 41개를 별도로 손으로 설계했다. 방식은 서로 다른 영역에서 가져온 동작과 대상을 억지로 엮는 것이다. 예를 들어 "cancel my calorie intake"(취소라는 동작과 칼로리 섭취량이라는, 코퍼스 어디에도 그런 조합이 없는 대상의 결합), "encrypt my sunrise times"(암호화라는 동작과 일출 시각의 결합) 같은 질의다. 이런 질의는 개별 단어("cancel", "calorie", "encrypt", "sunrise")는 코퍼스 어휘에 실제로 존재하기 때문에 순수 어휘 검색이 그럴듯한 상위 결과를 억지로 만들어낼 위험이 있다. 바로 이 점이 이 질의들을 "쉬운 네거티브"가 아니라 "하드 네거티브"로 만드는 이유다.

4.3 재현성과 라이선스

코퍼스 생성, 질의 생성, 보정/평가 분할에 쓰인 난수 시드를 전부 고정하고 이 논문에 명시했다(코퍼스 시드 42, 긍정 질의 시드 7, 하드 네거티브 시드 8, 분할 시드 11). 코퍼스와 질의는 이 논문을 위해 처음부터 새로 만든 합성 데이터이며, 어떤 실제 사용자 데이터나 제3자 소유 도구 목록도 포함하지 않는다. 생성 방법(영역-동작-대상 목록, 문장 틀, 동의어 사전, 순환 오프셋 규칙)은 이 논문의 본문에 전부 서술했으므로, 동일한 절차를 그대로 재구현하면 동일한 코퍼스와 질의 집합을 얻을 수 있다. 이 방법 자체를 별도 허가 없이 자유롭게 재사용, 변형, 재배포할 수 있는 공개된 방법으로 제공한다.

5 결과

5.1 전체 성능 비교

세 검색기를 120개의 평가용 긍정 질의에 대해 실측한 결과는 다음과 같다.

검색기	Recall@1	Recall@5	MRR
BM25 (어휘)	0.508	0.742	0.611
TF-IDF 문자 n-gram (유사-의미)	0.517	0.700	0.603
하이브리드 RRF	0.508	0.683	0.603

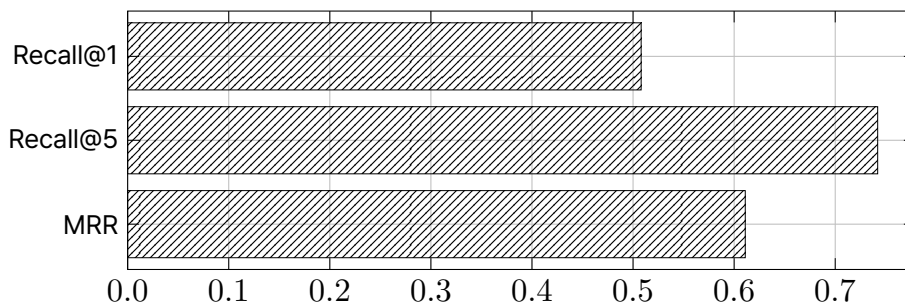


그림 1: 검색기별 Recall@1 / Recall@5 / MRR

가장 먼저 눈에 띄는 것은 하이브리드 RRF가 세 지표 중 어느 것에서도 단일 검색기보다 뚜렷하게 앞서지 않는다는 점이다. Recall@1에서는 BM25와 정확히 동률(0.508)이고 TF-IDF보다 0.009포인트 낮다. Recall@5와 MRR에서는 오히려 두 단일 검색기보다 낮다. 이 결과는 이 논문이 시작할 때 세운 가설, 즉 "하이브리드가 각 단일 검색기보다 상위 호환일 것"이라는 통념과 어긋난다. 원인을 뜯어보면 납득할 만하다. 이 실험의 질의는 (1) 순수 동의어 치환으로 만들어져 BM25가 잡기 어려운 경우와, (2) 원래 단어를 그대로 쓴 경우가 섞여 있고, TF-IDF 문자 n-gram은 동의어 치환에는 취약하지만 원래 단어를 그대로 쓰거나 형태만 살짝 바뀐 경우에는 강하다. 두 검색기가 서로 다른 하위 집합에서 강점을 보이는 상황에서, RRF처럼 순위만 보고 동등하게 융합하는 방식은 한쪽이 정답을 1위로 정확히 짚어낸 질의에서도 다른 쪽이 순위를 낮게 매기면 융합 순위가 끌려 내려가는 효과를 낼 수 있다. 즉 "두 검색기 중 하나라도 잘 맞으면 최종 결과도 좋아진다"는 직관은 순위 기반 융합에서는 자동으로 보장되지 않는다.

추가로 강건성을 확인하기 위해 긍정 질의 표본을 뽑는 난수 시드를 세 가지(7, 101, 202)로 바꿔가며 같은 실험을 반복했다. Recall@1 기준으로 하이브리드가 최고 성능 단일 검색기를 앞선 경우는 세 번 중 두 번(101번 시드에서 0.425로 BM25 0.408·TF-IDF 0.417보다 근소 우위, 202번 시드에서 0.417로 BM25 0.383보다 우위이나 TF-IDF 0.425보다는 낮음)이었고, 원 시드(7번)에서는 동률이었다. 세 번 모두 하이브리드가 압도적으로 앞선 경우는 없었다. 즉 이 실험 설계에서 하이브리드의 이득이 있다면 그것은 “항상, 크게”가 아니라 “가끔, 작게”에 가깝다.

5.2 오답 회피(기권) 성능

두 번째로 중요한 결과는 하드 네거티브 41개에 대한 기권 성능이다. 보정 집합에서 정답 질의의 약 90%가 통과하도록 정한 점수 게이트 임계값을, 정답이 없어야 하는 41개 질의에 그대로 적용했을 때 올바르게 기권한(즉 1위 점수가 임계값 아래로 떨어져 “모르겠다”고 판단한) 비율은 다음과 같다.

검색기	하드 네거티브 기권율	긍정 질의 오기권율
BM25	0.268	0.108
TF-IDF 문자 n-gram	0.317	0.133
하이브리드 RRF	0.122	0.042

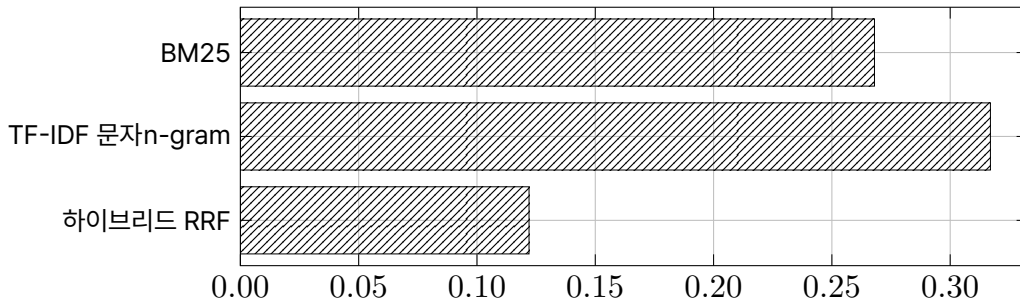


그림 2: 하드 네거티브 기권율 vs 긍정 질의 오기권율

이 표는 이 논문에서 가장 정직하게 보고해야 할 결과다. 하이브리드 RRF는 정답이 없는 질의의 87.8%(1 - 0.122)에 대해 여전히 무언가를 자신 있게 1위로 내놓았다. 이는 BM25(73.2%)나 TF-IDF(68.3%)보다 나쁜 수치다. 반면 하이브리드는 진짜 정답이 있는 질의를 잘못 기권하는 비율은 가장 낮다(4.2%, BM25는 10.8%, TF-IDF는 13.3%). 즉 하이브리드 RRF의 점수 게이트는 “관대한” 방향으로 치우쳐 있다. 정답을 놓치는 실수는 적지만, 정답이 없는데도 무언가를 내놓는 실수는 더 잦다.

이 비대칭의 원인은 RRF가 절대 점수 크기를 완전히 버리고 순위만 쓴다는 설계 자체에 있다. BM25나 TF-IDF는 관련성이 낮은 질의에 대해서는 1위 문서라 하더라도 점수 자체가 낮게 나오는 경향이 있어(질의 항이 코퍼스에 흔한 단어이거나 부분 매칭만 되는 경우), 점수 게이트가 그 낮은 점수를 잡아낼 여지가 있다. 반면 RRF 점수는 “1위이기만 하면” 상수 \$k\$에 의해 정해지는 값에 가까운 점수를 받는다. 두 검색기 중 단 하나에서라도 어떤 문서가 근소하게 1위를 차지하면, 그 절대적 관련성이 얼마나 약한지와 무관하게 RRF 융합 점수는 상대적으로 높게 나온다. 결과적으로 RRF의 점수 게이트는 개별 검색기의 점수 게이트보다 분별력이 떨어진다. 이것은 하이브리드 검색이 순위 품질(어떤 문서가 상위에 오는가)과 신뢰도 보정(그 상위 결과를 얼마나 믿어야 하는가)이라는 두 가지 서로 다른 목표를 동시에 잘 풀지 못할 수 있다는, 실무에 곧바로 적용 가능한 시사점이다.

6 한계와 캐비엣

이 논문의 결론은 이 특정 실험 설계 안에서만 유효하며, 아래 한계를 넘어서는 일반화는 저자도 지지하지 않는다.

첫째, 코퍼스가 합성이며 템플릿 기반이다. 20개 영역, 영역당 10개 동작과 10개 대상이라는 조합적 구조는 실제 스킬 생태계의 자연스러운 불균형(어떤 영역은 압도적으로 많은 도구를 갖고, 설명 문장의 길이와 구체성이 도구마다 크게 다른 현상)을 반영하지 못한다. 실제 도구 설명은 이 논문의 다섯 개 문장 틀보다 훨씬 다양하고, 때로는 부정확하거나 모호하게 쓰인다.

둘째, “의미 기반” 검색기가 진짜 의미 임베딩이 아니다. 문자 n-gram TF-IDF는 형태론적 변형과 부분 문자열 겹침을 잡아 내지만, “항공권을 찾아줘”와 “비행 스케줄을 알아봐줘”처럼 표면 형태가 완전히 다른 진짜 동의어 관계는 포착하지 못한다. 사전학습된 의미 임베딩 모델(다국어 문장 인코더 등)을 썼다면 이 논문의 핵심 결론, 특히 “하이브리드가 항상 이기지는 않는다”

는 결과가 바뀔 가능성을 배제할 수 없다. 이 실험 환경이 사전학습 모델 다운로드가 막힌 네트워크 조건에 있었기 때문에 나온 불가피한 선택이며, 이 논문은 이를 숨기지 않고 명시한다.

셋째, RRF의 파라미터($k=60$, 융합 깊이 50)를 튜닝하지 않고 문헌에서 흔히 쓰이는 기본값을 그대로 썼다. k 값을 낮추면 상위 순위의 영향력이 커지고, 융합 방식을 순위 기반이 아니라 정규화된 점수 기반(예: 각 검색기 점수를 $[0,1]$ 로 정규화한 뒤 가중합)으로 바꾸면 4절에서 관찰한 기권 성능 역전이 사라질 수도 있다. 이 논문은 “RRF라는 특정 융합 방식이 이 특정 설정에서 기권 성능을 해쳤다”는 것을 보였을 뿐, “모든 형태의 하이브리드 검색이 기권에 불리하다”는 훨씬 강한 주장은 하지 않는다.

넷째, 점수 게이트 임계값을 보정한 표본이 60개로 작다. 백분위수 추정은 표본이 작을수록 분산이 크므로, 다른 보정 표본을 뽑았다면 임계값과 그에 따른 기권율이 이 논문에 보고된 값과 다소 달라질 수 있다. 3절에서 언급한 다중 시드 재확인인 검색 성능 지표에는 적용했지만 기권 성능까지 전 시드에 대해 반복하지는 않았다는 점도 밝혀둔다.

다섯째, 코퍼스 규모(2000개 항목)와 질의 수(221개)는 이 논문의 제목이 말하는 “1000개 이상”이라는 조건은 충족하지만, 실제 초대형 마켓플레이스형 에이전트 생태계(수만 개 이상의 도구)에서 같은 패턴이 유지되는지는 이 논문만으로 확인할 수 없다. 코퍼스 크기가 커질수록 어휘 충돌(서로 다른 도구가 비슷한 단어를 쓰는 경우)이 늘어나 검색 난도가 오히려 급격히 올라갈 가능성이 있다.

7 데이터와 재현

이 실험은 다음 절차로 그대로 재현할 수 있다. (1) 3.1절에 서술한 20개 영역과 각 영역의 동작/대상 목록, 다섯 개 문장 틀을 그대로 구현하고 시드 42로 고정된 순서 섞기를 적용해 2000개 항목의 코퍼스를 만든다. (2) 3.2절에 서술한 동의어 치환 규칙(75% 확률로 사전에 정의한 동의어로 치환)과 여섯 개 질문 틀을 시드 7로 적용해 180개의 긍정 질의를 만들고, 시드 11로 60개(보정)와 120개(평가)로 분할한다. (3) 41개의 손으로 설계한 하드 네거티브 질의(서로 다른 영역의 동작-대상을 교차시킨 것)를 시드 8로 질문 틀에 적용한다. (4) 2.1절의 BM25 공식($k_1=1.5$, $b=0.75$, 어간 추출 없는 소문자 영숫자 토큰화)과 2.2절의 문자 n -gram(3~5, 단어 경계 인식) TF-IDF 코사인 유사도, 2.3절의 RRF($k=60$, 융합 깊이 50)를 구현한다. (5) 2.4절의 방식대로 보정 집합의 1위 점수 하위 10번째 백분위수를 임계값으로 잡고, 평가 집합과 하드 네거티브 집합에 적용해 Recall@1, Recall@5, MRR, 기권율을 계산한다. 이 절차에는 사전학습 모델이나 외부 API 호출이 전혀 필요하지 않으며, 코퍼스 생성부터 지표 계산까지 표준 수치 연산 라이브러리만으로 수 초 안에 끝난다. 이 방법은 자유롭게 재사용·변형·재배포할 수 있는 공개된 방법으로 제공하며, 저자 소속 조직이나 특정 소프트웨어 제품에 대한 배타적 권리를 주장하지 않는다.

8 참고문헌

1. Robertson, S. E., Walker, S., Jones, S., Hancock-Beaulieu, M. M., Gatford, M. (1994). Okapi at TREC-3. Proceedings of the Third Text REtrieval Conference (TREC-3).
2. Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333-389.
3. Salton, G., Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513-523.
4. Cormack, G. V., Clarke, C. L. A., Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. Proceedings of the 32nd International ACM SIGIR Conference, 758-759.
5. Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.
6. Patil, S. G., Zhang, T., Wang, X., Gonzalez, J. E. (2023). Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.
7. Qin, Y., et al. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv:2307.16789.

모든 실험은 고정 시드로 재현 가능하게 설계했고, 성능 판정은 코드가 직접 계산한 지표(Recall@1, Recall@5, MRR, 점수 게이트 기권율)로만 확정했다. 알려진 약점(RRF의 순위 전용 융합이 만드는 느슨한 기권 게이트)과 알려진 범위 제한(합성 템플릿 코퍼스, 진짜 의미 임베딩의 부재)은 결과와 함께 그대로 기록했다.